

Source Code Verification

Angela Gross // CSCI-441 // Project 2

VERTEX SHADER

```
//////////////////////////////////////

// Uniforms
// -----
uniform vec3 lightPosition;

//////////////////////////////////////

// Attributes
// -----
attribute vec4 tangent;

//////////////////////////////////////

// Varyings
// -----
varying vec2 vUv;
varying vec3 tsNormal;
varying vec3 tsPosition;
varying vec3 tsLightPosition;
varying vec3 tsEyePosition;

//////////////////////////////////////

// =====
// MAIN METHOD
// =====
void main()
{
    // Transforming from model, to world, and to projected view coordinates
    vec4 vertex = vec4( position, 1.0 );
    gl_Position = projectionMatrix * modelViewMatrix * vertex;

    // Update uv and convert object, light, and camera position to model-view.
    vUv = uv;
    vec3 vPosition = vec3(modelViewMatrix * vertex);
    vec3 vLightPosition = vec3(modelViewMatrix * vec4(lightPosition, 1.0));
    vec3 vCameraPosition = vec3(modelViewMatrix * vec4(cameraPosition, 1.0));

    // Compute tangent space
    vec3 vNormal = normalize(normal);
    vec3 vTangent = normalize(tangent.xyz);
    vec3 vBinormal = normalize( cross(vNormal, vTangent) * tangent.w );
    mat3 TBN = mat3(vTangent, vBinormal, vNormal);

    // Updating tangent space varying-variables
    tsNormal = TBN * vNormal;
    tsPosition = TBN * vPosition;
    tsLightPosition = TBN * vLightPosition;
    tsEyePosition = TBN * (vCameraPosition - vPosition);
}

//////////////////////////////////////
```

Commented [AG1]: Retrieved from *TerrainScene.js*

Commented [AG2]: Computed and inserted by Three JS

Commented [AG3]: Passed on to fragment shader since we need everything in tangent space

Commented [AG4]: This is fairly basic; just converting from object to view coordinates

Commented [AG5]: We pass on the texel coordinate to the fragment shader while converting these vectors to world (model) coordinates, as stressed by [Siddharth Hedge from GamaSutra](#) and Matyas Premez in his paper [Iterative Parallax Mapping with Slope Information](#). It needs to also be converted to view or the lighting will not change as the geometry moves.

Commented [AG6]: Normalizes each part, as we do not need the scalar part of the T, B, and N vectors to define the tangent space basis. This is also asserted by [Siddharth Hedge](#).

Commented [AG7]: Converts the vectors to tangent space by multiplying the basis by each of the vectors. This is supported by [Siddharth Hedge](#) and [Matyas Premez](#)

Commented [AG8]: The eye vector is found by subtracting the camera by the vertex's position, as asserted by the OpenGL tutorial by Typhoon Labs, [Chapter: 4 Advanced Shader](#).

FRAGMENT SHADER

//

// **Uniforms**

```
// -----  
uniform sampler2D colorTex;  
uniform sampler2D normalTex;  
uniform sampler2D dispTex;  
uniform float scale;  
uniform float bias;  
uniform int useNormal;  
uniform int useParallax;  
uniform vec3 lightPosition;  
uniform float lightIntensity;  
uniform vec3 lightColor;  
uniform vec3 ambientColor;
```

//

// **Varying**

```
// -----  
varying vec2 vUv;  
varying vec3 tsNormal;  
varying vec3 tsPosition;  
varying vec3 tsLightPosition;  
varying vec3 tsEyePosition;
```

//

// =====
// COMBINING NORMALS
// -----

// Since we have a normal from the geometry and the normal from the
// normal map, we need a way to combine both of them.
// From <http://blog.selfshadow.com/publications/blending-in-detail/>
// =====

```
vec3 blendWhiteout(vec3 n1, vec3 n2)  
{  
    vec3 r = vec3(n1.xy + n2.xy, n1.z*n2.z);  
    return normalize(r);  
}
```

//

// =====
// MAIN METHOD
// =====

void main()

```
{  
    // VARIABLES  
    // fNormal, fUv, and color change depending if parallax/normal are selected.  
    vec3 fNormal = tsNormal;  
    vec2 fUv = vUv;  
    vec4 color = texture2D(colorTex, vUv);
```

// **PARALLAX**
// Use scale, bias, and height to displace UV.

```
if(useParallax == 1)  
{  
    float height = texture2D(dispTex, vUv).a * scale + bias;
```

Commented [AG9]: All are given by *TerrainScene.js* except the light and ambient colors, as those are hard-coded in *TerrainShader.js*

Commented [AG10]: Passed on from the vertex shader.

Commented [AG11]: As the comment suggests, the “blend whiteout” method of combining two normals in tangent space from Colin Barré-Brisebois and Stephen Hill in *Blending in Detail*.

Commented [AG12]: As the comments suggest, these variables are declared at the top because they change depending on whether or not the parallax/normal options are selected. The normal and texel coordinates obviously change from normal and parallax mapping, respectively, but the color from the texture changes if the texel coordinate changes. Thus, these are all the default values for this focal variables.

Commented [AG13]: All of these operations for parallax mapping matches the ones given from the [paper by Matyas Premecz](#) in the section on parallax mapping.

```

        vec2 eyeVector = normalize(tsEyePosition.xy);
        fUv = vUv + ( eyeVector.xy * height);
        color = texture2D(colorTex, fUv);
    }

    // NORMAL
    // Retrieve normal from normal map and convert to be in range [-1, 1],
    // and combine with the pre-existing geometry normal.
    if(useNormal == 1)
    {
        vec3 normalMap = texture2D(normalTex, fUv).rgb;
        normalMap = normalize(normalMap * 2.0 - 1.0);
        fNormal = blendWhiteout(tsNormal, normalMap);
    }

    // LIGHTS (Lambertian-Directional and Ambient)
    // Using the Lights information from tangent space, compute a Lambertian,
    // directional light.
    vec3 lightDirection = tsLightPosition - tsPosition;
    vec3 lightVector = normalize(lightDirection);
    vec3 diffuse = (lightColor * lightIntensity) * max(dot(fNormal, lightVector), 0.0);
    vec3 ambient = ambientColor * lightIntensity;
    vec3 intensity = ambient + diffuse;

    gl_FragColor = vec4(color.rgb * intensity, color.a);
}

////////////////////////////////////

```

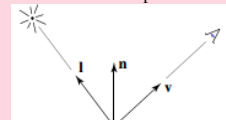
Commented [AG14]: Update the color with the new texel

Commented [AG15]: These operations (aside from the normal combination) are supported by the OpenGL tutorial, [Chapter: 4 Advanced Shader](#), [Matyas Premecz](#), and [Matt DesLaueriers' GitHub wiki tutorial](#), [Lesson 6: Normal Mapping](#).

Commented [AG16]: fUv is used, and so the change that the parallax mapping would make (if enabled) would affect the normal that is selected. This makes sense intuitively, and is also backed up by the [OpenGL tutorial](#).

Commented [AG17]: Combines the normal found from the geometry that was converted to tangent space, tsNormal, and the normal found from the normal map, normalMap. They are both normalized going in and out of the calculation, and so they will produce correct results when combined.

Commented [AG18]: The light direction is found by subtracting the light and the vertex positions (and they are in tangent space since the normal is in tangent space), and then it is normalized. This is supported by [Matt DesLaueriers' GitHub wiki tutorial](#), [Lesson 6: Normal Mapping](#), and we can see this in this photo:



The light's position subtracted by the object's results in the desired vector, l .

Commented [AG19]: The Lambertian-directional light (which gives a diffuse effect) is given by [Matt DesLaueriers' GitHub wiki tutorial](#), [Lesson 6: Normal Mapping](#), (the attenuation factor is taken out), along with [Edward Angel's text](#), [Interactive Computer Graphics](#), from 6.3.2: Diffuse Reflection. The dot product is maxed over 0.0 to prevent negative values of the diffuse light, as it'll be negative if the light source is below the horizon (From Angel's book as well)

Commented [AG20]: The ambient light is given by [Matt DesLaueriers' GitHub wiki tutorial](#), [Lesson 6: Normal Mapping](#), along with [Edward Angel's text](#), [Interactive Computer Graphics](#), from 6.3.1: Ambient Reflection.

Commented [AG21]: We can add these together to get both light contributions, which is given by [Matt DesLaueriers' GitHub wiki tutorial](#), [Lesson 6: Normal Mapping](#)

Commented [AG22]: The color of the texture is multiplied by the light's contributions, and this is given by [Matt DesLaueriers' GitHub wiki tutorial](#), [Lesson 6: Normal Mapping](#) along with the OpenGL tutorial, [Chapter: 4 Advanced Shader](#). Since the light contribution is only a 3-component vector, the alpha channel has to be manually added to the fragment color's 4-component vector.